

Django

TomerBu

נושאים להיום:

עמודי

Login/Logout/Registration

JWT

Throttling

ניהול שגיאות מרכזי

Filters/Search/Ordering

CORS

Pagination

משתני סביבה

יצירת משתמים:

serializers.py

```
from rest_framework.serializers import ModelSerializer
from django.contrib.auth.models import User

class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {
            'password': {'write_only': False, 'required': True},
            'id': {'read_only': True},
            'email': {'required': True},
            'username': {'required': True, 'min_length': 3},
        }

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user
```

views.py

```
class UsersViewSet(ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
```

נסיים בשיעור הבא :-)

urls.py

```
from .views import UsersViewSet
router.register('auth', UsersViewSet, basename='auth')
```

יצירת משתמש:

serializers.py

```
from rest_framework.serializers import ModelSerializer
from django.contrib.auth.models import User

class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {
            'password': {'write_only': True, 'required': True},
            'id': {'read_only': True},
            'email': {'required': True},
            'username': {'required': True, 'min_length': 3},
        }

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user

    def update(self, instance: User, validated_data):
        # instance is a user object
        # validated_data is a dictionary from the request
        # נחלץ את הסיסמא מהמילון של הערכים המאומתים
        password = validated_data.pop('password', None)
        # לולאה על כל הערכים במילון והוספה שלהם לאובייקט
        for key, value in validated_data.items():
            setattr(instance, key, value)

        instance.set_password(password)
        instance.save()
        return instance
```

ביצירת משתמש - סיסמא מוצפנת

בעדכון משתמש - סיסמא מוצפנת

שימוש בpip וvenv

אחרי כל הורדת ספרייה - או בסוף היום:

יוצר קובץ עם כל הספריות בשימוש בפרוייקט: `pip freeze > requirements.txt`

אחרי הורדה מגיטהב:

```
python -m venv .venv  
./venv/Scripts/activate
```

יצירת סביבה וירטואלית לפיתוח:

1

```
pip install -r requirements.txt
```

התקנת כל הספריות מקובץ requirements.txt

2

גישה לניהול משתמשים - רק לאדמין

מיד נטפל בהרשמה והתחברות

```
from rest_framework.permissions import DjangoModelPermissions, IsAdminUser
from core.permissions import IsOwnerOrModelPermissions

class UsersViewSet(ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [IsAdminUser]
```



גישה לניהול משתמשים - רק לאדמין

ViewSet
להרשמה והתחברות

```
from rest_framework.viewsets import ViewSet
from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework.decorators import action

class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]

    @action(detail = False, methods=['post', 'get'])
    def register(self, request):
        pass
```

מתודה עם שם לבחירתנו

(חובה לציין @action כדי שהראוטר יזהה אותה)

גישה לניהול משתמשים - רק לאדמין

ViewSet
להרשמה והתחברות

```
from rest_framework.viewsets import ModelViewSet, ViewSet

from rest_framework.response import Response
from django.contrib.auth.models import User
from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework.decorators import action
from rest_framework.authtoken.models import Token

class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]

    @action(detail = False, methods=['post', 'get'])
    def register(self, request):
        serializer = UserSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        user = serializer.save()

        token, _ = Token.objects.get_or_create(user=user)
        return Response({"token": token.key})
```


גישה לניהול משתמשים - רק לאדמין

urls.py

```
from blog.views import AuthViewSet  
router.register('auth', AuthViewSet, basename='auth')
```

גישה לניהול משתמשים - רק לאדמין

views.py

```
from rest_framework.permissions import IsAuthenticated
from django.contrib.auth import login, logout
class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]

    @action(detail = False, methods=['post', 'get'], permission_classes=[IsAuthenticated])
    def logout(self, request):
        try:
            logout(request)
            request.user.auth_token.delete()
        except:
            pass

        return Response({"message": "Logged out successfully"})
```

Views להרשמה והתחברות

views.py

```
from rest_framework.permissions import IsAuthenticated
from django.contrib.auth import login, logout
from rest_framework.authtoken.serializers import AuthTokenSerializer
class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [AllowAny]
```

```
@action(detail = False, methods=['post', 'get'])
def login(self, request):
    serializer = AuthTokenSerializer(data=request.data, context = {'request': request})
    serializer.is_valid(raise_exception=True)
    user = serializer.validated_data['user']
    token, _ = Token.objects.get_or_create(user=user)
    login(request, user)
    return Response({"token": token.key})
```

```
def list(self, request):
    return Response({
        "login": reverse('auth-login', request=request),
        "register": reverse('auth-register', request=request),
        "logout": reverse('auth-logout', request=request),
    })
```

ולידציות: פרטני לתכונה ולכל האובייקט:

serializers.py

```
class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {
            'password': {'write_only': False, 'required': True},
            'id': {'read_only': True},
            'email': {'required': True},
            'username': {'required': True, 'min_length': 3},
        }

    def validate_password(self, value):
        if len(value) < 8:
            raise ValidationError('Password must be at least 8 characters long.')
        return value

    def validate(self, attrs):
        if attrs['password'] == attrs['username']:
            raise ValidationError('Password must be different from username.')
        return super().validate(attrs)
```

JWT - Json Web Tokens

יש לנו מערכת שעובדת היטב:

חסרונות:

*לכל כניסה של משתמש לוגין -> המערכת צריכה לבדוק את Token מול הדטה-בייס

* נרצה לצרף מידע לToken (בתוך token)

Custom Claims

```
pip install django-rest-framework-simplejwt
```

https://django-rest-framework-simplejwt.readthedocs.io/en/latest/getting_started.html

Simple JWT

latest

Search docs

Getting started

Requirements

Installation

Cryptographic Dependencies (Optional)

Project Configuration

Usage

Docs » Getting started

Edit on GitHub

Getting started

Requirements

- Python (3.8, 3.9, 3.10, 3.11)
- Django (3.2, 4.0, 4.1, 4.2)
- Django REST Framework (3.10, 3.11, 3.12, 3.13, 3.14)

These are the officially supported python and package versions. Other versions will probably work

JWT - Json Web Tokens

```
pip install djangorestframework-simplejwt
```

```
REST_FRAMEWORK = {  
    'DEFAULT_RENDERER_CLASSES': [  
        'rest_framework.renderers.JSONRenderer',  
        'rest_framework.renderers.BrowsableAPIRenderer',  
    ],  
  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.AllowAny',  
    ],  
  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.SessionAuthentication',  
        'rest_framework.authentication.BasicAuthentication',  
        'rest_framework.authentication.TokenAuthentication',  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
}
```

JWT - Json Web Tokens

lec4/urls.py

```
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns += [
    path('api/v1/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/v1/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```



http://localhost:8000/api/v1/token/

POST



http://localhost:8000/api/v1/token/

Params Authorization Headers (10) **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

```
1  {
2    "username": "itomers",
3    "password": "123456"
4  }
```


JWT - Json Web Tokens

```
POST /api/v1/token/
```

```
HTTP 200 OK
```

```
Allow: POST, OPTIONS
```

```
Content-Type: application/json
```

```
Vary: Accept
```

```
{
```

```
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXV
```

```
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXV
```

```
}
```

משתמש יושב בתחנת רכבת ומוציא בקשת HTTPS

בראוטר ציבורי

הוא שולח את אותו **JWT** בכל בקשה

אם נרצה מנגנון מורחב:

ה**JWT** יהיה קצר מועד - למשל **30** שניות

ניתן לשלוח בקשה נוספת לקבלת **JWT** על סמך ה**JWT** הקיים

ואז בכל הבקשות יהיה **JWT** שונה

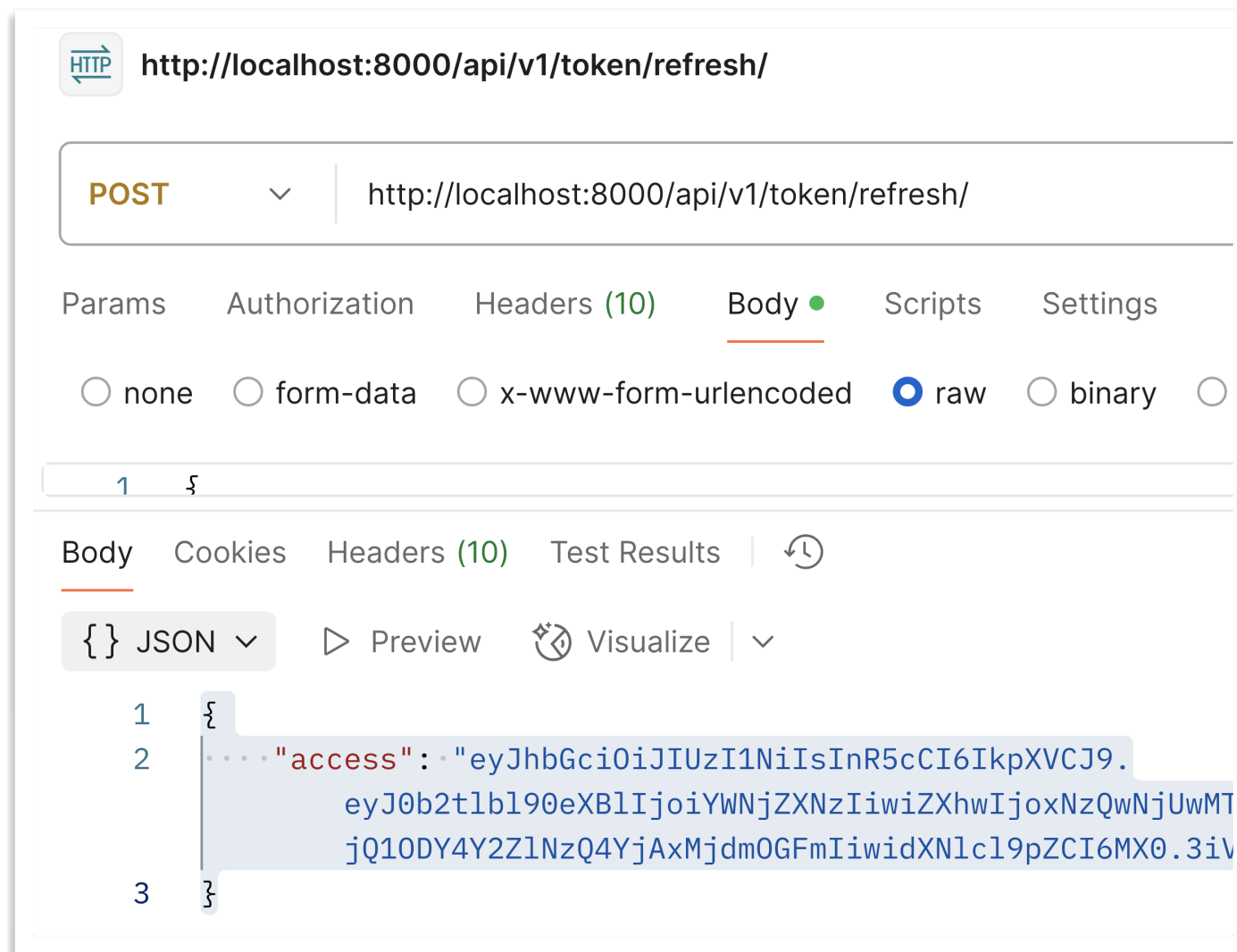
JWT - Json Web Tokens

לקוח מבצע לוגין: מקבל JWT קצר מועד ומקבל REFRESH_TOKEN ארוך טווח

הלקוח מבקש משאב כמו posts עם JWT ומקבל תגובה חיובית

אחרי 5 דקות - הלקוח רוצה לשלוח בקשה נוספת

מנגנון להחלפת JWT:



JWT - Json Web Tokens

settings.py

הגדרות ברירת מחדל:

```
from datetime import timedelta

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=5),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),
    "ROTATE_REFRESH_TOKENS": False,
    "BLACKLIST_AFTER_ROTATION": False,
    "UPDATE_LAST_LOGIN": False,
}
```

JWT - Json Web Tokens

settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'rest_framework_simplejwt',  
    'rest_framework_simplejwt.token_blacklist',  
    'taggit',  
    'hw',  
    'blog.apps.BlogConfig',  
]
```

כדי לאפשר החלפה של refresh tokens

המערכת שומרת את הtokens הישנים
כדי לא לאפשר לחזור עליהם

python manage.py migrate

```
SIMPLE_JWT = {  
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=1),  
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),  
    "ROTATE_REFRESH_TOKENS": True, #after refresh – get a new refresh token  
    "BLACKLIST_AFTER_ROTATION": True, #cancel old refresh tokens  
}
```

JWT - Json Web Tokens

settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'rest_framework_simplejwt',  
    'rest_framework_simplejwt.token_blacklist',  
    'taggit',  
    'hw',  
    'blog.apps.BlogConfig',  
]
```

כדי לאפשר החלפה של refresh tokens

המערכת שומרת את הtokens הישנים
כדי לא לאפשר לחזור עליהם

python manage.py migrate

```
SIMPLE_JWT = {  
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=1),  
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),  
    "ROTATE_REFRESH_TOKENS": True, #after refresh – get a new refresh token  
    "BLACKLIST_AFTER_ROTATION": True, #cancel old refresh tokens  
}
```

JWT - Json Web Tokens

בעת התחברות - נרצה להפיק JWT
וכנל לגבי הרשמה

אפשר היה לדלג על זה ולהשתמש בapi/v1/token
שהגדרנו בשלב הקודם

views.py

```
from core.authentication import get_tokens_for_user
```

```
@action(detail = False, methods=['post', 'get'])
def login(self, request):
    serializer = AuthTokenSerializer(data=request.data, context = {'request': request})
    serializer.is_valid(raise_exception=True)
    user = serializer.validated_data['user']
    token, _ = Token.objects.get_or_create(user=user)
    # from core.authentication import get_tokens_for_user
    jwt = get_tokens_for_user(user)
    login(request, user)
    return Response({"token": token.key, 'jwt': jwt})
```

JWT - Json Web Tokens

core/authentication.py

```
from rest_framework_simplejwt.tokens import RefreshToken

def get_tokens_for_user(user):
    refresh = RefreshToken.for_user(user)

    return {
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    }
```

views.py

```
from core.authentication import get_tokens_for_user
```

```
@action(detail = False, methods=['post', 'get'])
def login(self, request):
    serializer = AuthTokenSerializer(data=request.data, context = {'request': request})
    serializer.is_valid(raise_exception=True)
    user = serializer.validated_data['user']
    token, _ = Token.objects.get_or_create(user=user)
    # from core.authentication import get_tokens_for_user
    jwt = get_tokens_for_user(user)
    login(request, user)
    return Response({"token": token.key, 'jwt': jwt})
```

JWT - Json Web Tokens Custom Claims

הוספת מידע בתוך הJWT שיהיה נגיש ללקוח
(לא מידע רגיש)

https://django-rest-framework-simplejwt.readthedocs.io/en/latest/customizing_token_claims.html

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

class MyTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['name'] = user.name
        # ...

        return token
```

```
# Django project settings.py
...

SIMPLE_JWT = {
    # It will work instead of the default serializer(TokenObtainPairSerializer).
    "TOKEN_OBTAIN_SERIALIZER": "my_app.serializers.MyTokenObtainPairSerializer",
    # ...
}
```


JWT - Json Web Tokens

Custom Claims

הוספת מידע בתוך הJWT שיהיה נגיש ללקוח
(לא מידע רגיש)

blog/serializers.py

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer

class BlogTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['name'] = user.username
        token['isadmin'] = user.is_superuser

        return token
```

דוגמא לשימוש:

settings.py

BlogTokenObtainPairSerializer.get_token(user)

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=5),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),
    "ROTATE_REFRESH_TOKENS": True, #after refresh – get a new refresh token
    "BLACKLIST_AFTER_ROTATION": True, #cancel old refresh tokens
    "TOKEN_OBTAIN_SERIALIZER": "blog.serializers.BlogTokenObtainPairSerializer",
}
```


JWT - Json Web Tokens

Custom Claims



http://localhost:8000/api/v1/token/

POST



http://localhost:8000/api/v1/token/

Params

Authorization

Headers (8)

Body ●

Script

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw

```
1 {  
2   "username": "itomers",  
3   "password": "123456"  
4 }
```

Decoded

EDIT THE PAYLOAD AND SECRET

jwt.io

HEADER: ALGORITHM & TOKEN TYPE

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

PAYLOAD: DATA

```
{  
  "exp": 1740656243,  
  "iat": 1740655943,  
  "jti": "1d6e146c325a4135aab5c3239e69ce93",  
  "user_id": 1,  
  "name": "itomers",  
  "isadmin": true  
}
```

JWT - Json Web Tokens

Custom Claims

core/authentication.py

```
# from rest_framework_simplejwt.tokens import RefreshToken
from blog.serializers import BlogTokenObtainPairSerializer

# function (methods reside in a class)
def get_tokens_for_user(user):
    refresh = BlogTokenObtainPairSerializer.get_token(user)

    return {
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    }
```

login/register

משתמשים בפונקציה הזאת כדי להפיק JWT

ולכן הם מקבלים את הערך המעודכן

DRF - Throttling

DRF מאפשרת לנו להגביל את כמות הבקשות מלקוחות:

*הגבלת כמות הכניסות ע"י משתמש לא רשום
*הגבלת כמות הכניסות ע"י משתמש רשום
*הגבלה בהתאמה אישית

את ההגבלות ניתן לבצע ברמת ה View או ברמת כל האפליקציה.

חשוב לציין - הספירה מתבצעת לפי משתמש:

לפי כתובת IP (למשתמש אנונימי)
לפי מזהה משתמש (מחובר)

כלומר לקוח שיש לו 6 בקשות ליום:
יכול לבקש סה"כ 6 בקשות מכל ה Views יחד
ולא Per View

קיימת אפשרות להגביל גישה לפי Scope
ע"י יצירת מחלקה להתאמה אישית

<https://www.django-rest-framework.org/api-guide/throttling/>

DRF - Throttling

סקירה של אפשרויות Throttle:

הגבלת כניסה ברמת האפליקציה:

```
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/day',
        'user': '1000/day'
    }
}
```

הגבלת כניסה ברמת הView:

:Class Based View

```
from rest_framework.response import Response
from rest_framework.throttling import UserRateThrottle
from rest_framework.views import APIView

class ExampleView(APIView):
    throttle_classes = [UserRateThrottle]

    def get(self, request, format=None):
        content = {
            'status': 'request was permitted'
        }
        return Response(content)
```

הגבלת כניסה בFunction Based View:

```
@api_view(['GET'])
@throttle_classes([UserRateThrottle])
def example_view(request, format=None):
    content = {
        'status': 'request was permitted'
    }
    return Response(content)
```

הגבלת כניסה בaction:

```
@action(detail=True, methods=["post"], throttle_classes=[UserRateThrottle])
def example_adhoc_method(request, pk=None):
    content = {
        'status': 'request was permitted'
    }
    return Response(content)
```

DRF - Throttling

```
REST_FRAMEWORK = {  
    **REST_FRAMEWORK,  
    'DEFAULT_THROTTLE_CLASSES': [  
        'rest_framework.throttling.AnonRateThrottle',  
        'rest_framework.throttling.UserRateThrottle'  
    ],  
    'DEFAULT_THROTTLE_RATES': {  
        'anon': '1/min',  
        'user': '5/minute'  
    }  
}
```

DRF - Throttling

settings.py

```
'DEFAULT_THROTTLE_RATES': {  
    'anon': '1/min',  
    'user': '5/minute',  
  
    'list_posts_user': '5/minute',  
    'list_posts_anon': '1/minute',  
  
    'retrieve_post_user': '5/minute',  
    'retrieve_post_anon': '1/minute',  
}
```

DRF - Throttling

blog/throttling.py

```
from rest_framework.throttling import UserRateThrottle, AnonRateThrottle

class CreatePostUserRateThrottle(UserRateThrottle):
    scope = 'create_post_user'

class CreatePostAnonRateThrottle(AnonRateThrottle):
    scope = 'create_post_anon'

class ListPostsUserRateThrottle(UserRateThrottle):
    scope = 'list_posts_user'

class ListPostsAnonRateThrottle(AnonRateThrottle):
    scope = 'list_posts_anon'
```


DRF - Throttling

blog/throttling.py

```
from blog.throttling import (
    CreatePostUserRateThrottle,
    CreatePostAnonRateThrottle,
    ListPostsUserRateThrottle,
    ListPostsAnonRateThrottle
)
class PostsViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [IsOwnerOrModelPermissions]

    mapping = {
        'create': [CreatePostUserRateThrottle, CreatePostAnonRateThrottle],
        'list': [ListPostsUserRateThrottle, ListPostsAnonRateThrottle],
    }
    def get_throttles(self):
        throttles = self.mapping.get(self.action, [])
        return [throttle() for throttle in throttles]
```

(1) נייבא את כל המחלקות שלנו:

ניצור מילון שממפה לפעולות איזה throttle class הן צריכות:

נממש את המתודה get_throttles - מתודה מובנית:
ה action הוא שם המתודה

DRF - Throttling

blog/throttling.py

```
from rest_framework.throttling import BaseThrottle
from django.core.cache import cache
```

קאש הוא מילון עם **:timeout**

```
class BlogRateThrottle(BaseThrottle):
    def allow_request(self, request, view):
        if request.user.is_authenticated:
            user_id = f'user_{request.user.id}'
            max_requests = 5
        else:
            user_id = f"anon_{request.META.get('REMOTE_ADDR')}"
            max_requests = 1
```

אם המשתמש מחובר - ניקח את הuser_id

אם המשתמש מחובר - ניקח את הuser_ip

```
#path = /api/v1/posts/1
path = request.path
```

בכל מקרה - נבדוק את הendpoint

```
cache_key = f'throttle_{user_id}_{path}'
```

נגדיר מפתח לקאש:

```
hit_count = cache.get(cache_key, 0)
```

נבדוק בקאש - כמה כניסות כבר יש:
אחרת אפס - כלומר טרם נכנס לנתיב:

```
if hit_count > max_requests:
    return False
```

אם חרג - נחזיר False

```
else:
    cache.set(cache_key, hit_count + 1, timeout = 60)
    return True
```

נגדיל את כמות הצפיות ונחזיר True

DRF - Exception Handling

settings.py

```
REST_FRAMEWORK = {  
    'EXCEPTION_HANDLER': 'rest_framework.views.exception_handler',  
}
```



blog/exceptions.py

פונקציה מובנית שמטפלת בכל השגיאות שנזרקות

```
from rest_framework.views import exception_handler  
from rest_framework.response import Response  
from rest_framework import status  
from django.http import Http404  
from django.core.exceptions import PermissionDenied  
from rest_framework.exceptions import Throttled  
  
def blog_exception_handler(exc, context):  
    response = exception_handler(exc, context)  
  
    if isinstance(exc, Http404):  
        return Response({"error": "Resource Not Found!!!"}, status=status.HTTP_404_NOT_FOUND)  
  
    if isinstance(exc, ValueError):  
        return Response({"error": "Invalid Input!!!"}, status=status.HTTP_400_BAD_REQUEST)  
  
    if isinstance(exc, PermissionDenied):  
        return Response({"error": "Permission Denied!!!"}, status=status.HTTP_403_FORBIDDEN)  
  
    if isinstance(exc, Throttled):  
        return Response({"error": "Request was Throttled!!!"}, status=status.HTTP_429_TOO_MANY_REQUESTS)  
  
    return response
```

DRF - Exception Handling

settings.py

```
REST_FRAMEWORK = {  
    # 'EXCEPTION_HANDLER': 'rest_framework.views.exception_handler',  
    'EXCEPTION_HANDLER': 'blog.exceptions.blog_exception_handler',  
}
```

שימוש בפונקציה שיצרנו



הוספת יכולות מיון סינון וחיפוש:

To use `DjangoFilterBackend`, first install `django-filter`.

```
pip install django-filter
```

טרמינל:

Then add `'django_filters'` to Django's `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'django_filters',           settings.py  
    ...  
]
```

You should now either add the filter backend to your settings:

```
settings.py  
REST_FRAMEWORK = {  
    'DEFAULT_FILTER_BACKENDS': ['django_filters.rest_framework.DjangoFilterBackend']  
}
```

הוספת יכולות מיון סינון וחיפוש:

```
class PostsViewSet(ModelViewSet):  
    queryset = Post.objects.all()  
    serializer_class = PostSerializer  
    permission_classes = [IsOwnerOrModelPermissions]  
    filterset_fields = ['author', 'title', 'text']
```

הוספת יכולות מיון:

Ordering

username - ascending

username - descending

email - ascending

email - descending

```
from rest_framework.filters import OrderingFilter
from django_filters.rest_framework import DjangoFilterBackend
```

```
class PostsViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [IsOwnerOrModelPermissions]
    filter_backends = [OrderingFilter, DjangoFilterBackend]

    filterset_fields = ['author', 'title', 'text', 'created_at', 'updated_at']
```

הוספת יכולות חיפוש:

```
from rest_framework.filters import OrderingFilter, SearchFilter
from django_filters.rest_framework import DjangoFilterBackend

class PostsViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [IsOwnerOrModelPermissions]
    filter_backends = [OrderingFilter, DjangoFilterBackend, SearchFilter]

    filterset_fields = ['author', 'title', 'text', 'created_at', 'updated_at']
    search_fields = ['title', 'text']
```

שדה החיפוש - מכיל את המילה
ללא חשיבות לupper/lowercase

Prefix	Lookup
^	<code>startswith</code> Starts-with search.
=	<code>iexact</code> Exact matches.
\$	<code>iregex</code> Regex search.
@	<code>search</code> Full-text search (Currently only supported Django's PostgreSQL backend).
None	<code>icontains</code> Contains search (Default).

For example:

```
search_fields = ['=username', '=email']
```


CORS

```
pip install django-cors-headers
```

settings.py

```
INSTALLED_APPS = [  
    ...  
    # CORS  
    'corsheaders',  
]
```

settings.py

```
MIDDLEWARE = [  
    ...  
    # CORS  
    'corsheaders.middleware.CorsMiddleware',  
    'django.middleware.common.CommonMiddleware',  
]
```

settings.py

```
CORS_ALLOW_ALL_ORIGINS = False  
CORS_ALLOWED_ORIGINS = (  
    'http://localhost:8081',  
)
```

`CorsMiddleware` should be placed as high as possible, especially before any middleware that can generate responses such as Django's `CommonMiddleware` or Whitenoise's `WhiteNoiseMiddleware`. If it is not before, it will not be able to add the CORS headers to these responses.

CORS:

```
pip install django-cors-headers
```

settings.py

```
CORS_ORIGIN_ALLOW_ALL = False

CORS_ALLOWED_ORIGINS = [
    "https://example.com",
    "https://sub.example.com",
    "http://localhost:8080",
    "http://127.0.0.1:9000",
    "http://127.0.0.1:3000",
    "http://127.0.0.1:5173",
]
```

כאן נכלול את הכתובות של צד הלקוח שלנו.

```
pip freeze > requirements.txt
```

Pagination

Limit Offset Pagination

```
REST_FRAMEWORK = {  
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.LimitOffsetPagination',  
    'PAGE_SIZE': 1  
}
```

<http://localhost:8000/api/v1/posts/?limit=3&offset=3>

מקבל 2 פרמטרים Limit ו-Offset
כמו בSQL

User Profile List

GET /blog/api/profiles/

HTTP 200 OK

Allow: GET, POST, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
{  
  "count": 2,  
  "next": null,  
  "previous": null,  
  "results": [  
    {  
      "id": 1,  
      "bio": "This is my bio",  
      "created_at": "2025-02-19T14:27:34.405334Z",  
      "updated_at": "2025-02-19T16:27:01.463387Z",  
      "profile_picture": "http://127.0.0.1:8000/media/profile_pictures/gravatar.jpeg",  
      "user": 2  
    },  
    {  
      "id": 2,  
      "bio": "A cool guy",  
      "created_at": "2025-02-19T16:17:38.156072Z",  
      "updated_at": "2025-02-19T16:17:38.156098Z",  
      "profile_picture": null,  
      "user": 1  
    }  
  ]  
}
```

Pagination

Page Number Pagination

```
REST_FRAMEWORK = {  
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',  
    'PAGE_SIZE': 1  
}
```

לא מאפשר למשתמש להגדיר את הLimit
רק את מספר העמוד:
?page=2

User Profile List

<http://127.0.0.1:8000/blog/api/profiles/?page=2>

GET /blog/api/profiles/

HTTP 200 OK

Allow: GET, POST, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
{  
  "count": 2,  
  "next": "http://127.0.0.1:8000/blog/api/profiles/?page=2",  
  "previous": null,  
  "results": [  
    {  
      "id": 1,  
      "bio": "This is my bio",  
      "created_at": "2025-02-19T14:27:34.405334Z",  
      "updated_at": "2025-02-19T16:27:01.463387Z",  
      "profile_picture": "http://127.0.0.1:8000/media/profile_pictures/gravatar.jpeg",  
      "user": 2  
    }  
  ]  
}
```

More Pagination Options

ניתן לבחור את סוג ה-Pagination גם פרטני ל-ViewSet מסוים:

```
from rest_framework.pagination import PageNumberPagination

class UserProfileViewSet(ModelViewSet):
    queryset = UserProfile.objects.all()
    serializer_class = UserProfileSerializer
    pagination_class = PageNumberPagination
```

אם רוצים שליטה גם על מספר התוצאות בעמוד יש להגדיר מחלקה שיורשת מ-PageNumberPagination

נראה בעמוד הבא:

More Pagination Options

שליטה על כמות התוצאות בעמוד:

blog/pagination.py

מחלקה שיורשת PageNumberPagination

```
from rest_framework.pagination import PageNumberPagination
```

```
class LargeResultsSetPagination(PageNumberPagination):
    page_size = 25
    page_size_query_param = 'page_size'
    max_page_size = 100
```

ברירת מחדל של page_size היא None - כלומר משתמש לא יכול לשלוט על כמות התוצאות אלא אם נגדיר את הערך

```
class StandardResultsSetPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 25
```

views.py

```
from rest_framework.pagination import PageNumberPagination
```

```
class UserProfileViewSet(ModelViewSet):
    queryset = UserProfile.objects.all()
    serializer_class = UserProfileSerializer
    pagination_class = StandardResultsSetPagination
```

More Pagination Options

בשימוש במחלקה שיצרנו יש לנו כעת 2 פרמטרים לשליטה על התוצאות:

http://localhost:8000/api/v1/posts/?page=2&page_size=2

מספר העמוד

כמות התוצאות בעמוד
(מוגבל ל-25 במחלקה שכתבנו)

More Pagination Options

blog/pagination.py

הוספה של מידע לתגובת ה pagination

```
from rest_framework.pagination import PageNumberPagination
from rest_framework.response import Response
import math

class BlogPagination(PageNumberPagination):
    page_size_query_param = 'page_size' # Allows clients to set page size
    max_page_size = 25 # Prevents excessively large pages

    def get_paginated_response(self, data):
        # account for the query parameter
        page_size_mod = self.request.query_params.get('page_size')

        if page_size_mod and page_size_mod.isdigit():
            self.page_size = int(page_size_mod)

        # Calculate total pages – total items divided by page size
        # If there are any remaining items, add an extra page
        total_pages = math.ceil(self.page.paginator.count / self.page_size)
        return Response({
            "total_results": self.page.paginator.count, # Total items
            "total_pages": total_pages, # Total pages
            "current_page": self.page.number, # Current page number
            "results_per_page": self.page_size, # Page size
            "next": self.get_next_link(),
            "previous": self.get_previous_link(),
            "results": data # Actual results
        })
```


More Pagination Options

settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_PAGINATION_CLASS': 'blog.pagination.BlogPagination',  
    'PAGE_SIZE': 1  
}
```

http://localhost:8000/api/v1/posts/?page=2&page_size=2

מספר העמוד

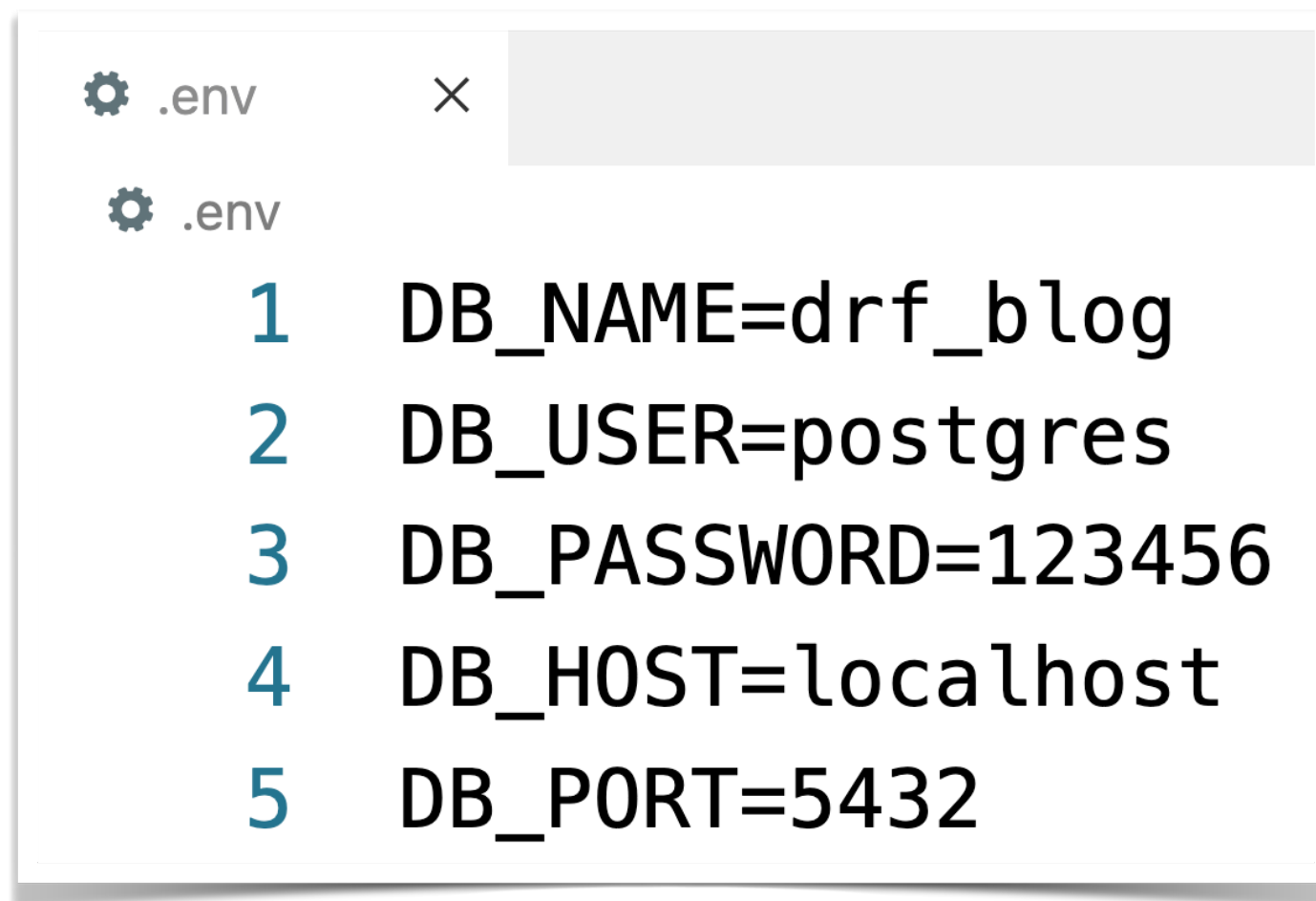
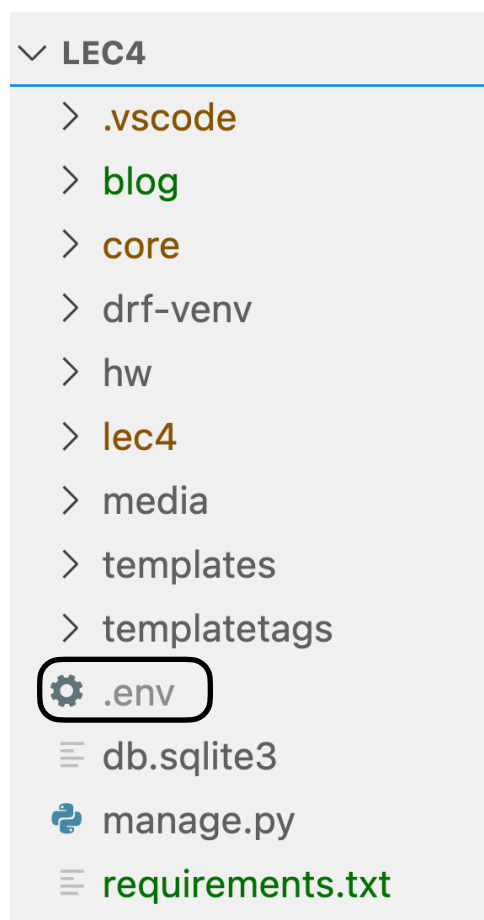
כמות התוצאות בעמוד
(מוגבל ל-25 במחלקה שכתבנו)

```
{  
  "total_results": 9,  
  "total_pages": 5,  
  "current_page": 2,  
  "results_per_page": 2,  
  "next": "http://localhost:8000/api/v1/posts/?page=3&page_size=2",  
  "previous": "http://localhost:8000/api/v1/posts/?page_size=2",  
  "results": [  
    {  
      "id": 3,  
      "tags": [  
        "API",  
        "Python"  
      ],  
      "author_id": 1,  
      "title": "Title for Post 3",  
      "text": "Text for Post 3",  
      "created_at": "2025-02-23T09:13:10.876621Z",  
      "updated_at": "2025-02-23T09:13:10.876635Z",  
      "status": "draft"  
    },  
    {  
      "id": 4,  
      "tags": [],  
      "author_id": 1,  
      "title": "title for post 4",  
      "text": "text for post 4",  
      "created_at": "2025-02-23T09:18:14.245481Z",  
      "updated_at": "2025-02-23T09:18:14.245493Z",  
      "status": "draft"  
    }  
  ]  
}
```


עבודה עם משתני סביבה:

אפשר לטעון משתני סביבה
עם הספרייה `python-decouple`

1. ניצור קובץ בתיקיה הראשית בשם `.env`.



(נוסיף את הקובץ `gitignore` שלנו כדי שלא יעלה לגיטהאב)

עבודה עם משתני סביבה:

```
pip install python-decouple
```

2

settings.py

שימוש במשתני הסביבה שהגדרנו בקובץ .env

```
from decouple import config
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': config('DB_NAME'),
        'USER': config('DB_USER'),
        'PASSWORD': config('DB_PASSWORD'),
        'HOST': config('DB_HOST'),
        'PORT': config('DB_PORT'),
    }
}
```

3

אפשר לבצע גם המרה אוטומטית לסוג המשתנה הרצוי באופן הבא - אם יש צורך:

4

```
PORT = config('PORT', cast=int)
IS_HAPPY = config('IS_HAPPY', cast=bool)
```

עבודה עם venv:

הקפאה:

```
pip freeze > requirements.txt
```

שחזור:

```
python -m venv .venv
```

```
./venv/scripts/activate
```

```
pip install -r requirements.txt
```